

APLSuite: An Integrated Suite for CD4+ T Cell Epitope Prediction via Antigen Processing Likelihood

Jiarui Li¹, Marco K. Carbullido¹, Jai Bansal², Samuel J. Landry³, Ramgopal R. Mettu^{1,*},

1 Department of Computer Science, Tulane University, New Orleans, LA, United States

2 Isidore Newman School, New Orleans, LA, United States

3 Department of Biochemistry and Molecular Biology, Tulane University School of Medicine, New Orleans, LA, United States

* rmettu@tulane.edu

Abstract

Computational epitope prediction is a critical tool for exploring and understanding CD4+ T cell-mediated immune responses, a key aspect of adaptive immunity. While existing computational methods primarily focus on supervised learning approaches, they often overlook the essential role of antigen processing in determining binding specificity. To address this limitation, our group developed Antigen Processing Likelihood (APL), an algorithm that integrates crystallographic B-factor, solvent accessible surface area (SASA), hydrogen exchange protection factors (COREX), and sequence entropy.

In this paper we introduce APLSuite, a comprehensive and lightweight software suite designed to streamline APL-based epitope prediction. APLSuite integrates distributed RESTful API services, a Python client for data aggregation and processing, a data science tool for efficient epitope computation, and a user-friendly graphical user interface for non-coding users. It provides a seamless and efficient pipeline for APL calculation and epitope prediction that can be finished in minutes with GPU-acceleration, which has not been implemented by existed tools. This flexible and extensible software suite is deployable on desktop and cloud environments, offering both guided and customizable workflows to meet diverse research needs in immunology research and immunotherapy development.

Introduction

CD4+ T cells are critical to adaptive immunity, playing a central role in mediating immune responses across diverse biological contexts [1]. The computational prediction of antigenic peptides that are presented and bound to T cells has been an area of research for decades, which highly contributes to immunology research and immunotherapy development. While current approaches primarily utilize supervised learning techniques (e.g., NetMHC-II [2,3] provided by IEDB [4,5]) to predict peptide-MHC-II binding, these methods typically do not account for the role of antigen processing.

To address this limitation, our group developed the Antigen Processing Likelihood (APL) algorithm [6–8], which models conformation stability using crystallographic B-factor (or AlphaFold pLDDT), solvent accessible surface area (SASA), correlations with hydrogen exchange protection factors (COREX) [9], and sequence entropy. Despite its robust theoretical framework, APL relies on multiple computational

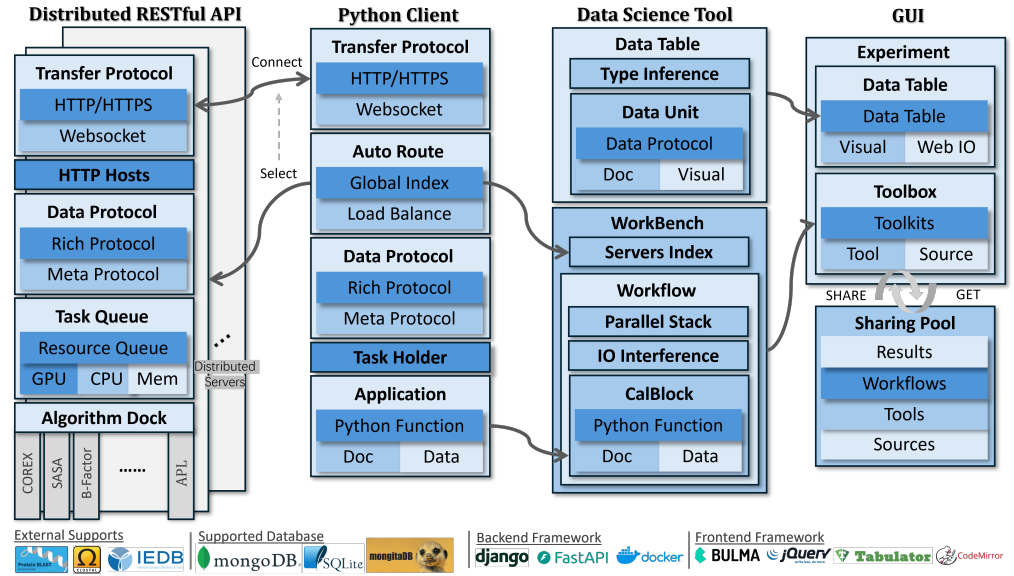


Fig 1. APLSuite Framework Overview: The APLSuite framework including Distributed RESTful API (DRAF), Python client, data science tool, and graphical user interface (GUI). The APL components and itself are developed as API endpoints, remote Python function, and web UI based tool using this framework. It is network service based on Django, FastAPI, and Docker. The storage engine supporting SQLite, MongoDB, and MongitaDB. The computational services are also supported by BLAST [15,16], Clustal Omega [17–19], and IEDB [4,5].

components, which currently lack integration into a single tool. As a result, researchers must compute each factor manually using separate tools and then combine the results using the APL algorithm.

For SASA calculations, FreeSASA [10] is the commonly used Python package. COREX is available through a web service [11], but it is a single-processor, CPU-based algorithm, consuming hours to days for a single calculation. To overcome this bottleneck, our group developed both CPU-parallelized and GPU-parallelized versions of COREX [12–14], which reduce computation times from hours or days to under 180 seconds. Sequence entropy calculations depend on BLAST [15,16] and Clustal Omega [17–19] for sequence alignment, which require additional computational resources such as GPUs, CPUs, and access to large-scale databases.

These tools are provided by different sources, operate in varying formats, and demand diverse computational resources. To address these challenges, we present an integrated software suite that we call **APLSuite** (Fig 1) that can run the entire APL pipeline seamless within minutes. It consists of a distributed RESTful API framework, a Python client for distributed API access, a data science tool, and a web application graphical user interface for non-coding users. The suite is designed to be flexible, extensible, and lightweight, enabling deployment on both desktop systems and cloud services. For users, the web front-end provides a step-by-step guided mode for ease of use, as well as a highly customizable mode for advanced applications. Leveraging the framework, we provided several APL and APL associated with MHC prediction automated pipelines, which accepting PDB file, PDB ID, and AlphaFold predicted mmCIF file as inputs separately.

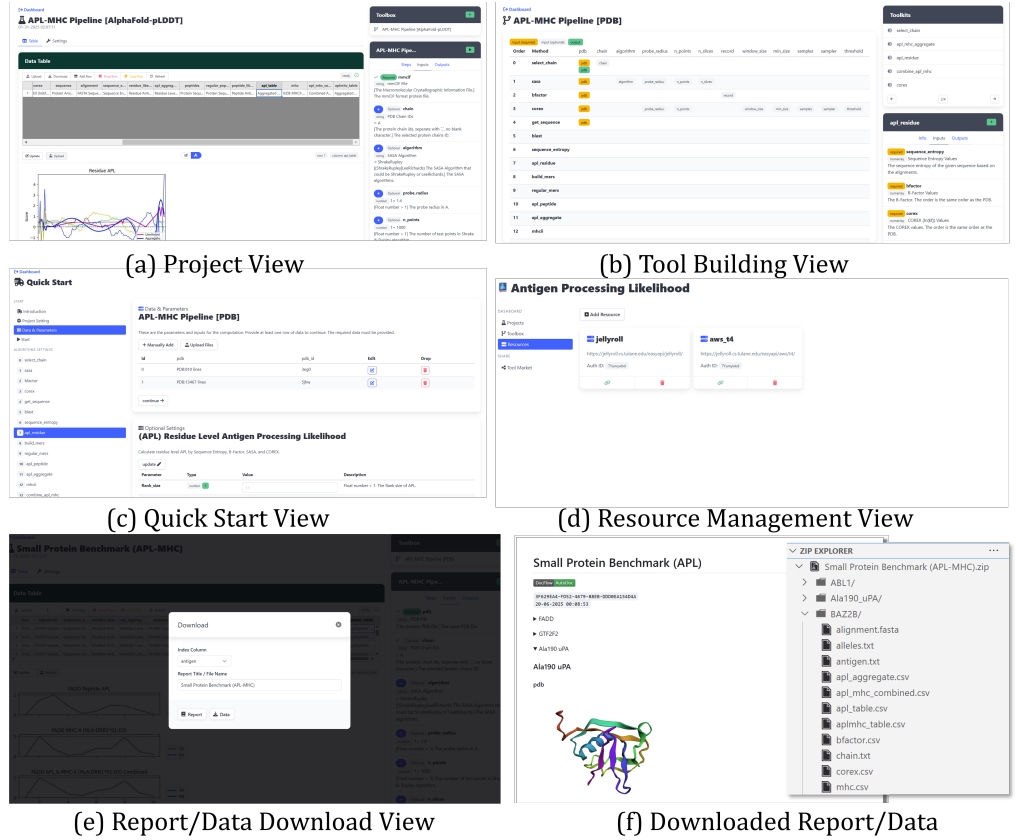


Fig 2. Graphical User Interface (GUI): The views of GUI including project view, tool view, quick start view, and resource management view. Which enables both step-by-step guided running and highly flexible workbench. It also supports to download the results as a report or a compressed file of CSV tables.

Design and Implementation

This application framework consists of four components including: Graphical User Interface (GUI) for non-coding users, Distributed RESTful API Framework (DRAF), Python client, and Data Science Tool (DST) to support the GUI and coding users.

Graphical User Interface

To accommodate non-coding users and simplify the utilization of APL computations, we have developed a Graphical User Interface (GUI) supported by the DRAF and DST frameworks, built on Django [20]. The GUI consists of five primary views: dashboard view, project view, tool view, quick start view, and resource management view. Each computation algorithm or chained algorithm is represented as a tool, while each computation case is managed as a project. Users can manage tools and projects centrally through the dashboard view.

The project view (Fig 1.(a)) is designed for managing and executing specific computational tasks (projects). It includes four main components: data table, data visualization, toolbox, and job list. The editable data table stores computational inputs (e.g., PDB files, PDB IDs), parameters (e.g., COREX thresholds, APL weights), and outputs. Users can edit the table directly by modifying values or uploading files to

customize inputs or parameters. For any selected cell in the data table, the corresponding visualizations generated by DST are displayed, offering immediate feedback on the data. The toolbox lists all tools available for the current project (e.g., COREX, SASA, APL). Users can explore each tool’s algorithm combinations, descriptions, inputs, and outputs. If parameter customization is required, users can add specific parameters to the data table by clicking the “Add” button. Once configured, users can execute the tool by clicking “Run”, which queues the task in the job list. The job list tracks all tasks, displaying their status (e.g., in-queue, running, error) and providing specific error messages when applicable. This view offers researchers a fully customizable and flexible interface for running individual components or the entire APL pipeline with their preferred parameters.

For users seeking a simplified experience, the quick start view (Fig 2.(c)) provides a step-by-step guided interface. Users are prompted to input or upload required files (e.g., PDB files for the APL pipeline). Computations can then be initiated directly. While parameter customization is supported, it is optional and can be skipped, making this view ideal for users who prioritize simplicity over customization.

The tool view (Fig 2.(b)) allows users to build or customize tools and tool chains. Users can assemble their own tools by combining existing blocks, ensuring that the inputs and outputs of adjacent blocks share the same name. The system automatically infers required inputs and outputs from the assembled blocks, providing a preview for user confirmation. This feature empowers users to create tailored workflows that meet their specific requirements.

The resource management view (Fig 2.(d)) enables users to manage computational resources efficiently. As supported by the DST framework, users can connect to multiple servers, with the system automatically indexing the optimal server for executing algorithms. This ensures efficient resource utilization across distributed environments.

Distributed RESTful API Framework

The Distributed RESTful API Framework (DRAF) as shown in Fig 3 is designed using the FastAPI framework [21], enabling seamless transformation of standard Python functions into RESTful API endpoints with auto-generated documentation. DRAF incorporates several essential features to enhance efficiency, flexibility, and scalability:

- Automatic conversion of Python functions into RESTful API endpoints.
- Automatic generation of endpoint documentation based on function parameters and descriptions.
- A parameter fingerprint-based caching mechanism to eliminate redundant computations.
- A resource-aware job scheduling system that optimizes the execution of computational tasks.
- A standardized data type protocol for consistent data exchange between APIs, servers, and clients, facilitating algorithm integration.
- Dual network communication support for standard HTTP requests and high-frequency WebSocket interactions.

```

1  @register(required_resources={'cpu':1, 'cuda':0})
2  @cache(disable=False)
3  def sasa(pdb: PDB['The input PDB file.'],
4          algorithm: SASAlgorithm['The SASA algorithms.'] = 'ShrakeRupley',
5          probe_radius: NumberGreaterThan1['The probe radius in A.'] = 1.4,
6          n_points: NumberGreaterThan1['The number of test points in Shrake & Rupley algorithm.'] = 1000,
7          n_slices: NumberGreaterThan1['Get the number of slices per atom in Lee & Richards algorithm.'] = 20,
8          resources={}) -> dict[
9      SASA['sasa', 'The solvent accessible surface area. The order is the same order as the PDB.']
10 ]:
11     """Solvent Accessible Surface Area
12     Calculate the solvent accessible surface area for the given protein.
13     The results will be an array concatenated by the order of sorted(chains).
14     """
15     sasa_values = _sasa(pdb,
16                        algorithm=algorithm,
17                        probe_radius=probe_radius,
18                        n_points=int(n_points),
19                        n_slices=int(n_slices),
20                        worker_num=1)
21     return dict(sasa=sasa_values)

```

Fig 3. Distributed RESTful API Framework (DRAF): The Distributed RESTful API Framework (DRAF) enables seamless transformation of standard Python functions into RESTful API endpoints with auto-generated documentation.

Standardized Data Format for Algorithm Integration: DRAF enforces all data to follow a data protocol based on meta-types and rich types, to allow validating input and output data efficiently and chaining algorithms by connecting the output of one algorithm to the compatible input of another. The meta-types include numbers, strings, and numeric arrays, and the rich types are defined as subsets of these meta-types, with custom rules, descriptions, and names. For instance, Protein Data Bank (PDB) files can be defined as a subset of strings. Moreover, it enables users to chain algorithms by connecting the output of one algorithm to the compatible input of another, facilitating the creation of complex, stacked workflows on the client side.

Automatic API Generation and Documentation: Unlike traditional RESTful API frameworks, DRAF leverages its standardized input and output definitions to simplify API creation. Developers can transform any Python function with typed parameters and return values into an API endpoint. Based on these type annotations and function comments, DRAF automatically generates comprehensive documentation without requiring additional code. The meta-type standardization also facilitates parameter serialization, enabling the creation of unique hash fingerprints for parameter combinations. These fingerprints allow efficient caching of results, accelerating query responses while ensuring user privacy, as cached results are stored in a secure, non-readable format.

Resource-Aware Job Scheduling: To accommodate the diverse computational requirements of different algorithms, DRAF employs a resource-dependent job scheduling system. Instead of executing tasks sequentially, DRAF organizes jobs into multiple queues, each optimized for specific computational resources. For example, a server equipped with two GPUs and ten CPUs could allocate one queue with two GPUs and five CPUs for GPU-intensive tasks (e.g., COREX), another queue with four CPUs for CPU-intensive tasks (e.g., BLAST), and a final queue with one CPU for lightweight tasks (e.g., SASA). This design minimizes runtime by assigning tasks to the most appropriate queue automatically.

WebSocket Support for Efficient Job Monitoring: A common challenge with asynchronous API services is the need for clients to repeatedly query job statuses using

HTTP requests, which can strain resources. To address this, DRAF supports WebSocket communication as an alternative. WebSockets establish a persistent connection between the client and server, allowing the client to monitor job progress without sending repeated HTTP requests. This reduces resource consumption and improves efficiency.

DRAF enables the transformation of Python functions with well-defined parameters into API endpoints that include automatic documentation, result caching, resource-aware job scheduling, and advanced communication protocols. Using DRAF, we have developed API endpoints for B-Factor, SASA, COREX, Sequence Entropy, BLAST, MHC-II prediction, and APL running on distributed servers.

Python Client and Data Science Tool

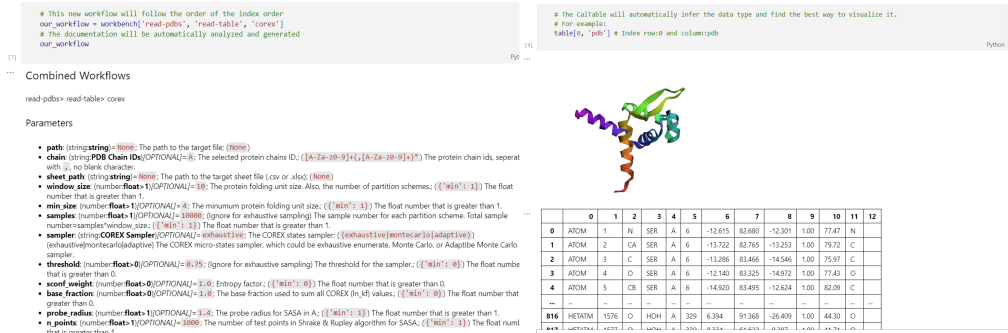


Fig 4. Data Science Tool (DST): Data Science Tool (DST) streamlines access to DRAF’s functionality and make it more accessible to a wide range of users

To fully leverage the capabilities of DRAF and enhance usability, we provide a Python client and a Data Science Tool (DST) as shown in Fig 4. These tools streamline access to DRAF’s functionality and make it more accessible to a wide range of users.

The Python client acts as a bridge between users and DRAF by automatically exporting all API endpoints of a server as Python functions. Each function corresponds to an API endpoint, with documentation dynamically constructed by querying the server. This approach simplifies API usage, as users can interact with DRAF’s functionality directly within Python without dealing with HTTP requests or manual endpoint configuration.

Built on top of the Python client, the Data Science Tool (DST) further enhances usability by enabling advanced operations and seamless integration with multiple servers. Key features of the DST include:

- **Server Indexing and Load Balancing:** Users can connect to multiple DRAF servers, and the DST automatically identifies the best server for a remote API call. This functionality is as intuitive as indexing an element in a Python list.
- **Algorithm Chaining:** Leveraging DRAF’s standardized data type protocol, the DST allows users to chain multiple algorithms by providing a sequence of API endpoint indices. This simplifies the creation of complex workflows. In addition, for user customized chained algorithms, DST will automatically generate a documentation based on the components’ documentation.
- **Data Table Structure:** The DST introduces a Pandas-like data structure, called the data table, which enables users to perform operations, calculations, and visualizations on data seamlessly. This familiar interface minimizes the learning curve for data manipulation.

- **Basic Visualization:** Using meta-type definitions, the DST supports built-in visualizations for the data table. Users can view graphical representations of individual table cells without writing any additional code. In addition, the protein related visualization extensions are embedded in the package for convenience.
- **Custom and Extensible Visualizations:** The DST supports user-defined visualizations and extensions, allowing users to tailor the tool to their specific needs. For example, rich visualizations such as 3D views of PDB protein structures or alignment views for FASTA sequences can be implemented or installed as extensions.

The Python client and DST work together to maximize the utility of DRAF by simplifying API access, enabling complex algorithmic workflows, and providing data manipulation and visualization capabilities. These tools empower users, ranging from novice coders to experienced researchers, to integrate and analyze computational results efficiently and effectively.

Results

To enhance the accessibility and utility of the APL computational pipelines, we integrated three categories of APL algorithms into the application: (1) fully automated APL pipelines, (2) APL pipelines incorporating MHC binding predictions (APL-MHC), and (3) standalone APL components. Both the APL and APL-MHC pipelines accept PDB files, PDB IDs, or AlphaFold-predicted mmCIF structures as inputs. These pipelines compute B-factors (or 100-pLDDT for AlphaFold mmCIF), solvent-accessible surface area (SASA), COREX stability, and sequence entropy. They subsequently generate residue- and peptide-level APL scores. The default parameters for APL aggregation is extracted from the CD4+ T cell immunodominance of influenza virus hemagglutinin research [22]. The APL-MHC pipeline further integrates MHC binding predictions from the IEDB API based on user-specified alleles, combining these scores with APL results. As examples, the small proteins benchmark computed by APL and APL-MHC pipelines are provided with the application as shown in Fig 2.e and Fig 2.f.

To support modular usage, we provide independent tools for B-factor, 100-pLDDT, SASA, COREX, sequence entropy, residue APL, peptide APL, and MHC prediction. These tools are managed by a default local DRAF service, which is automatically deployed upon installation. Additionally, an independent DRAF server is available for installation on separate machines, enabling users to integrate it into their local workflows.

Single Antigen Analysis: Example with a Varicella Zoster antigen

As an example of a typical use case for APLSuite analysis we consider the analysis of a single antigen: the glycoprotein E (gE) of Varicella Zoster Virus (VSV), a key antigen in the Shingrix vaccine [23]. Figure 5 shows APL and MHC predictions along with mapped epitopes.

We assessed APL, MHC-II binding (using seven high-performing alleles [24]), and a linear combination of both [25]. This particular workflow is preloaded in the GUI and can be set up and dispatched in seconds for a single antigen. After processing, results can be saved in tabular and PDF formats for further analysis or presentation. Figure 5 shows results both at the residue and peptide levels, along with comparisons to ground truth epitopes (known in this case, but not required as input).

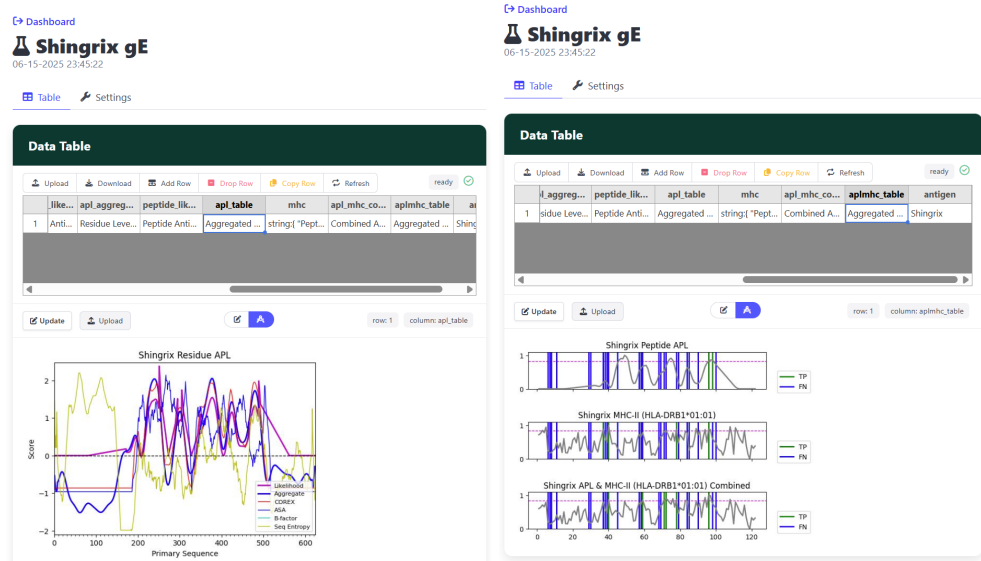


Fig 5. Application of the APL-MHC pipeline the VSV gE antigen.

Batch Processing: a Small Protein Benchmark

As part of the distribution we provide a benchmark of small proteins (from [13]). This dataset compiled 13 small proteins ranging in size from 63 to 117 residues. Here we discuss how APLSuite can be used to conduct batch processing. The GUI provided with APLSuite enables the computation of APL and MHC binding of these proteins by setting up a single project that can be dispatched on numerous resources. Figure 6 shows a snapshot of the generated data table and results for one antigen.

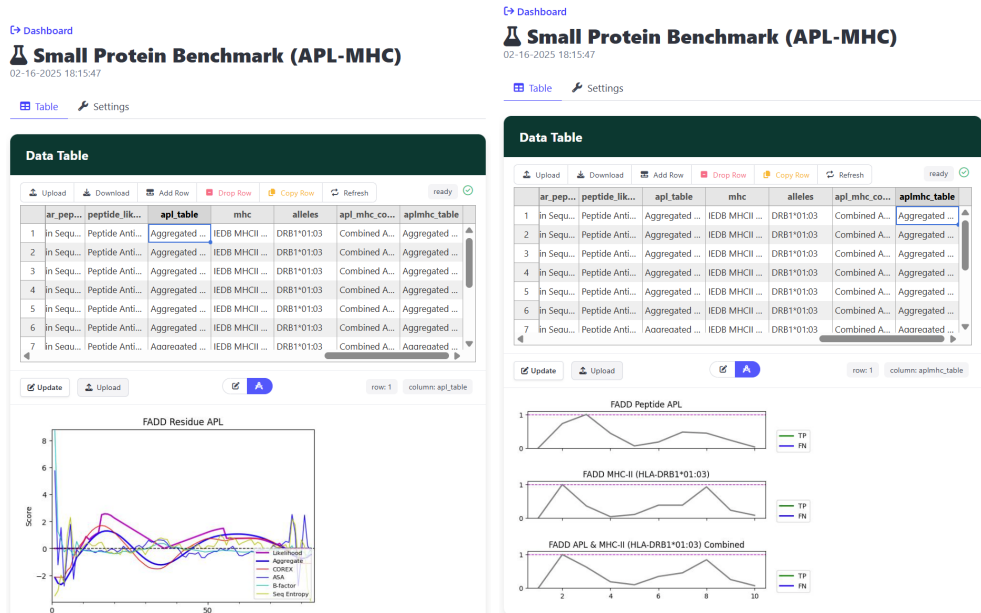


Fig 6. Application of the APL-MHC pipeline to a set of antigens.

Conclusion

APLSuite provides an integrated suite of tools, including the Distributed RESTful API Framework (DRAF), a Python client, a Data Science Tool (DST), and a graphical user interface (GUI). This suite enables seamless deployment of APL components across multiple servers with diverse computational resources, transforming Python functions into API endpoints with minimal effort. The web application and DST offer intuitive, table-like operations and automatic data visualization, catering to users with varying levels of coding expertise. From non-coding users and novice programmers to experienced researchers, APLSuite simplifies access to and utilization of the APL pipeline, facilitating efficient computational workflows.

Availability and Future Directions

APLSuite is implemented in Python 3.12 with Django and FastAPI frameworks in the environment with BLAST, Clustal Omega, and Docker installed (or BLAST API). It is released under a *GPLv3 license*. APLSuite can be downloaded from <https://github.com/Jiarui0923/APL> and run on any standard desktop computer or cloud services.

Acknowledgments

The authors thank the anonymous reviewers for their valuable suggestions. We also acknowledge support from from Harold L. and Heather E. Jurist Center of Excellence for Artificial Intelligence at Tulane University, and the Amazon AWS Cloud Credit for Research program for providing necessary GPU resources.

References

1. Kumar BV, Connors TJ, Farber DL. Human T cell development, localization, and function throughout life. *Immunity*. 2018;48(2):202–213.
2. Nilsson JB, Kaabinejadian S, Yari H, Kester MG, van Balen P, Hildebrand WH, et al. Accurate prediction of HLA class II antigen presentation across all loci using tailored data acquisition and refined machine learning. *Science Advances*. 2023;9(47):eadj6367.
3. Nilsson JB, Kaabinejadian S, Yari H, Peters B, Barra C, Gragert L, et al. Machine learning reveals limited contribution of trans-only encoded variants to the HLA-DQ immunopeptidome. *Communications biology*. 2023;6(1):442.
4. Vita R, Mahajan S, Overton JA, Dhanda SK, Martini S, Cantrell JR, et al. The immune epitope database (IEDB): 2018 update. *Nucleic acids research*. 2019;47(D1):D339–D343.
5. IEDB. [iedb.org](http://www.iedb.org/): free epitope database and prediction resource; 2024. Available from: <http://www.iedb.org/>.
6. Mettu RR, Charles T, Landry SJ. CD4+ T-cell epitope prediction using antigen processing constraints. *Journal of immunological methods*. 2016;432:72–81.
7. Charles T, Moss DL, Bhat P, Moore PW, Kummer NA, Bhattacharya A, et al. Cd4+ T-cell epitope prediction by combined analysis of antigen conformational

- flexibility and peptide-Mhcii binding affinity. *Biochemistry*. 2022;61(15):1585–1599.
8. Landry SJ, Mettu RR, Kolls JK, Aberle JH, Norton E, Zvezdaryk K, et al. Structural Framework for Analysis of CD4+ T-Cell Epitope Dominance in Viral Fusion Proteins. *Biochemistry*. 2023;62(17):2517–2529.
 9. Hilser VJ, Freire E. Structure-based calculation of the equilibrium folding pathway of proteins. Correlation with hydrogen exchange protection factors. *Journal of molecular biology*. 1996;262(5):756–772.
 10. Mitternacht S. FreeSASA: An open source C library for solvent accessible surface area calculations. *F1000Research*. 2016;5.
 11. Vertrees J, Barritt P, Whitten S, Hilser VJ. COREX/BEST server: a web browser-based program that calculates regional stability variations within protein structures. *Bioinformatics*. 2005;21(15):3318–3319.
 12. Bhattacharya A, Wrabl JO, Landry SJ, Mettu RR. Parallel Computation of Conformational Stability for CD4+ T-cell Epitope Prediction. In: 2023 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). IEEE; 2023. p. 88–93.
 13. Li J, Landry SJ, Mettu RR. GPU Acceleration of Conformational Stability Computation for CD4+ T-cell Epitope Prediction. In: 2024 IEEE International Conference on Bioinformatics and Biomedicine (BIBM). IEEE; 2024. p. 191–196.
 14. Li J, Landry SJ, Mettu RR. GPU Acceleration for Markov Chain Monte Carlo Sampling. In: 4th International Conference on AIML Systems (AIMLSystems 2024). ACM; 2024. p. 1–8.
 15. Altschul SF, Gish W, Miller W, Myers EW, Lipman DJ. Basic local alignment search tool. *Journal of molecular biology*. 1990;215(3):403–410.
 16. Camacho C, Coulouris G, Avagyan V, Ma N, Papadopoulos J, Bealer K, et al. BLAST+: architecture and applications. *BMC bioinformatics*. 2009;10:1–9.
 17. Edgar RC, Batzoglou S. Multiple sequence alignment. *Current opinion in structural biology*. 2006;16(3):368–373.
 18. Sievers F, Wilm A, Dineen D, Gibson TJ, Karplus K, Li W, et al. Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular systems biology*. 2011;7(1):539.
 19. Sievers F, Higgins DG. Clustal Omega for making accurate alignments of many protein sequences. *Protein Science*. 2018;27(1):135–145.
 20. Django. The web framework for perfectionists with deadlines.; 2024. Available from: <https://www.djangoproject.com/>.
 21. FastAPI. FastAPI framework, high performance, easy to learn, fast to code, ready for production; 2024. Available from: <https://fastapi.tiangolo.com/>.
 22. Cassotta A, Paparoditis P, Geiger R, Mettu RR, Landry SJ, Donati A, et al. Deciphering and predicting CD4+ T cell immunodominance of influenza virus hemagglutinin. *Journal of Experimental Medicine*. 2020;217(10):e20200206.

23. Voic H, de Vries RD, Sidney J, Rubiro P, Moore E, Phillips E, et al. Identification and characterization of CD4+ T cell epitopes after Shingrix vaccination. *Journal of Virology*. 2020;94(24):10–1128.
24. Paul S, Sidney J, Peters B, Sette A. Development and validation of a broad scheme for prediction of HLA class II restricted T cell epitopes. In: *Proceedings of the 5th ACM Conference on Bioinformatics, Computational Biology, and Health Informatics*; 2014. p. 733–738.
25. Bhattacharya A, Lyons MC, Landry SJ, Mettu RR. Incorporating antigen processing into CD4+ T cell epitope prediction with integer linear programming. In: *Proceedings of the 13th ACM International Conference on Bioinformatics, Computational Biology and Health Informatics*; 2022. p. 1–10.